



Estruturas de dados heterogêneas

A estrutura de dados é essencial na computação para organizar e manipular dados eficientemente. Em Linguagem C, o uso de ponteiros e alocação dinâmica de memória facilita a criação de estruturas de dados flexíveis. Dominar esses conceitos permite programar de forma organizada e eficiente, criando programas robustos e escaláveis para resolver problemas complexos e lidar com grandes volumes de dados.

Profa. Daisy Cristine Albuquerque da Silva

Preparação

Antes de iniciar o conteúdo, você precisa de um compilador , como o GCC, e uma IDE. Recomendamos que você instale o Dev-C++, que é um ambiente de desenvolvimento integrado livre que utiliza os compiladores do projeto GNU para compilar programas para o sistema operacional Microsoft Windows.

Para baixar o código-fonte dos exemplos, [clique aqui](#) ou na seção “Explore +” deste material.

Objetivos

- Empregar ponteiros com a utilização da linguagem de programação C.
- Definir estrutura de dados heterogênea.
- Aplicar structs com a utilização da linguagem de programação C.
- Empregar as estruturas de dados aninhadas, os vetores de estruturas e a instrução typedef usando a linguagem de programação C.

Introdução

Acompanhe neste vídeo o que são estruturas de dados nas suas formas homogênea e heterogênea.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Ponteiros e memória

Ponteiros são variáveis que armazenam endereços de memória, permitindo um acesso mais eficiente e flexível aos dados. Em C, a memória do programa é organizada em segmentos como Text, Data, Stack e Heap, cada um com funções específicas. A alocação de memória pode ser estática, automática ou dinâmica, sendo essencial para a gestão eficiente de recursos. A utilização de ponteiros, structs e arrays de structs possibilita a manipulação de estruturas de dados heterogêneas.

Conheça neste vídeo as principais regiões de memória manipuláveis por um programa em C, o conceito de ponteiro e sua importância prática.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição de ponteiro

Segundo Schildt (1996), ponteiro é uma variável que contém um endereço de memória.

Podemos ainda definir o ponteiro como um tipo especial de variável, na qual o valor atribuído é um endereço de memória. Ou seja, ponteiro ou apontador é uma variável capaz de armazenar um endereço de memória ou o endereço de outra variável.

Para entendermos melhor essa definição, precisamos compreender como a memória de um programa é organizada e o que é um endereço de memória.

Memória

É um componente do computador responsável pelo armazenamento de dados e instruções. Ela é composta por palavras, sendo cada palavra identificada unicamente a partir de um endereço, ou seja, um endereço de memória. Cada palavra tem uma capacidade de armazenamento da informação, isto é, uma quantidade de bytes que a palavra representa.

Uma memória é composta por palavras e toda palavra possui um endereço único, conforme é visto na Tabela 1.



Representação de bytes.

Endereço	Palavras
0 (00)	Palavra 0
1 (001)	Palavra 1
2 (010)	Palavra 2
3 (011)	Palavra 3
4 (100)	Palavra 4
5 (101)	Palavra 5
6 (110)	Palavra 6

Tabela 1: Representação da memória.
Daisy Albuquerque.

O endereço de memória de um processo em execução é dividido em vários segmentos lógicos. Destacamos alguns dos mais importantes a seguir.

Text

Contém o código do programa e suas constantes. Esse segmento é alocado durante a criação do processo ("exec") e permanece do mesmo tamanho durante toda a vida do processo.

Data

É a memória de trabalho do processo, na qual ficam alocadas as variáveis globais e estáticas. Tem tamanho fixo ao longo da execução do processo.

Stack

Contém a pilha de execução, na qual são armazenados os parâmetros, os endereços de retorno e as variáveis locais de funções. Pode variar de tamanho durante a execução do processo.

Heap

Contém blocos de memória alocadas dinamicamente, a pedido do processo, durante sua execução. Varia de tamanho durante a vida do processo.

Podemos observar na imagem a seguir a distribuição dos segmentos lógicos do endereço de memória.



Segmentos lógicos do endereço de memória.

Atividade 1

Questão

Considerando a definição de ponteiro na linguagem de programação C, qual das seguintes opções melhor descreve sua funcionalidade?

A

Um ponteiro armazena o valor de uma variável diretamente.

B

Um ponteiro pode apenas apontar para variáveis do tipo inteiro.

C

Um ponteiro armazena o endereço de memória de uma variável, permitindo acesso indireto ao valor armazenado nessa variável.

D

Um ponteiro é utilizado exclusivamente para operações matemáticas de alto desempenho.

E

Um ponteiro não pode ser modificado após sua inicialização.



A alternativa C está correta.

A definição de ponteiro em C é uma variável que armazena o endereço de memória de outra variável, possibilitando o acesso indireto ao valor armazenado nessa variável. As outras opções estão incorretas. Um ponteiro não armazena o valor diretamente, mas sim o endereço de memória em que o valor está armazenado (opção A). Ponteiros podem apontar para variáveis de qualquer tipo de dado, não apenas inteiros (opção B). Ponteiros não são utilizados exclusivamente para operações matemáticas, mas para uma gama de manipulações de memória (opção D). Ponteiros podem ser modificados após sua inicialização, permitindo que apontem para diferentes endereços de memória (opção E).

Alocação de memória

É um conceito fundamental na programação em C, importante para a gestão eficiente dos recursos. Existem três formas principais de alocação: estática, automática e dinâmica.

A **alocação estática**, usando o modificador static, mantém o valor das variáveis durante toda a execução do programa. A **alocação automática** ocorre quando variáveis locais são criadas e liberadas dentro do escopo de funções. A **alocação dinâmica**, por meio das funções malloc, free e realloc, permite a criação e o gerenciamento flexível de memória durante a execução do programa – essencial para o desenvolvimento de aplicações complexas e eficientes.

Entenda neste vídeo o conceito de alocação de memória em suas formas automática, estática e dinâmica, e acompanhe onde cada uma deve ser aplicada.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Agora que já sabemos o que é memória e como ela está organizada internamente, vamos entender as três formas de alocação de memória usando a linguagem C. São elas:

- Alocação estática (geralmente aloca em DATA).
- Alocação automática (geralmente aloca em STACK).
- Alocação dinâmica (geralmente aloca em HEAP).

Alocação estática

Ocorre a declaração de variáveis globais (alocadas fora de uma função) ou estáticas (alocadas dentro de uma função), usando o modificador **static**. Nesse caso, o valor alocado à variável se mantém durante toda a vida do programa, exceto quando é modificado explicitamente.

Analisemos o **Exemplo 1** para entendermos melhor a alocação de memória.

c

```
#include < stdio.h >

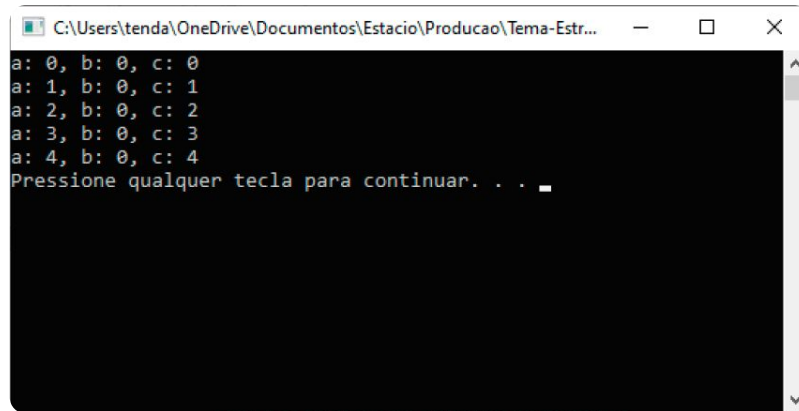
static int a = 0; // variável global, alocação estática

void incrementa(void)
{
    int b = 0; // variável local, alocação automática
    static int c = 0; // variável local, alocação estática

    printf ("a: %d, b: %d, c: %d\n", a, b, c);
    a++;
    b++;
    c++;
}

int main(void)
{
    int i;
    for (i = 0; i < 5; i++)
        incrementa();
    system("pause");
    return(0);
}
```

A execução desse código gera a seguinte saída, conforme mostra a imagem a seguir.



```
a: 0, b: 0, c: 0
a: 1, b: 0, c: 1
a: 2, b: 0, c: 2
a: 3, b: 0, c: 3
a: 4, b: 0, c: 4
Pressione qualquer tecla para continuar. . .
```

Saída da execução do Exemplo 1.

Analizando esse código, podemos observar nas linhas 3 e 8 a declaração da variável **a** e da variável **c**. A variável **a** foi declarada como uma variável global e a variável **c** como uma variável local.

Também é observado que as variáveis foram alocadas de forma estática e inicializadas uma única vez. Assim, seus valores se preservam entre as chamadas consecutivas da **função incrementa**.



Comentário

Na primeira chamada da função incrementa, a variável **a**, por ser global, já tem o valor 0 e a variável **c** recebe o valor 0 dentro da função. Na linha 10, esse valor é impresso, e logo em seguida, as variáveis são incrementadas, passando a possuir o valor 1. A função incrementa é mais uma vez chamada e podemos observar que as variáveis, por serem estáticas, permanecem com os seus valores, assim, é impresso o valor 1 e são incrementadas mais uma vez, passando a possuir o valor 2.

Nesse contexto, o programa continua a sua execução até o valor da variável **i** ser igual a 5, conforme o laço de repetição **FOR**.

Alocação automática

Ocorre a declaração de variáveis locais e parâmetros de função. Sendo assim, a alocação dessas variáveis é realizada quando a função é invocada e liberada quando a função termina.



Comentário

Voltando a observar o Exemplo 1, a variável **b** é declarada de forma automática ao ser invocada a função incrementa, conforme a linha 7 do código-fonte, e descartada quando a função se encerra. Nesse caso, o valor da variável não se preserva durante a chamada da função incrementa, isto é, o valor alocado à variável não se mantém durante toda a vida do programa.

Alocação dinâmica

Nesta alocação é requisitada explicitamente pelo programa uma área de memória para armazenamento de dados. O controle das áreas alocadas dinamicamente pode ser:

- Manual
- Semiautomático

Assim, o programa as utiliza e depois as libera quando não forem mais necessárias ou quando o programa encerrar. A liberação das áreas alocadas dinamicamente é realizada pelo programador.

A alocação dinâmica manual pode ser realizada de forma **simples** ou por **vetor**. Observe!

```
c
#include < stdlib.h >
void * malloc (size_t size)
```

A função **malloc** aloca determinado número de bytes na memória e retorna um ponteiro para o primeiro byte alocado. Se não for possível alocar, a função retorna **NULL**.

A liberação é realizada pela função **free**, que libera o espaço alocado, isto é, libera a área de memória previamente alocada dinamicamente. Acompanhe!

```
c
#include < stdlib.h >
void free (void *ptr)
```

A função **free** libera o número de bytes alocados previamente na memória, apontado pelo ponteiro **ptr**, que continua apontando para a área liberada. Por isso, é aconselhável mudar seu valor para **NULL** após a liberação. Entenda melhor!

```
c
ptr = malloc (1024);
...
free (ptr);
ptr = NULL; // não é obrigatório, mas aconselhável
```

O redimensionamento da área alocada é realizado pela função **realloc**, da seguinte forma:

```
c
#include < stdlib.h >
void * realloc (void *ptr, size_t newsize)
```

A função **realloc** redimensiona a área previamente alocada, apontada por **ptr**, para o novo tamanho **newsize**. A função retorna o novo endereço da área de memória, que pode ser diferente do anterior, caso tenha sido necessário mudar de lugar.

Em resumo, podemos visualizar cada função com seu objetivo na Tabela 2. Observe!

Função	Ação	Sintaxe
Função malloc	A função "malloc" aloca um determinado número de bytes na memória e retorna um ponteiro para o primeiro byte alocado. Se não for possível alocar, a função retorna NULL.	#include < stdlib.h > void *malloc (size_t size)

Função	Ação	Sintaxe
Função free	A função “free” libera o número de bytes alocados previamente na memória, apontado por “ptr”.	#include <stdlib.h > void free (void *ptr)
Função realloc	A função “realloc” redimensiona a área previamente alocada, apontada por “ptr”, para o novo tamanho “ <i>newsize</i> ”.	#include <stdlib.h > void *realloc (void *ptr, size_t newsize)

Tabela 2: Relação de funções utilizadas na alocação dinâmica simples de memória.
Daisy Albuquerque.

Na **alocação por vetor**, utilizam-se ponteiros para a alocação. Assim, basta especificar o tamanho desse vetor no momento da alocação. Após a alocação de uma área com vários elementos, ela pode ser acessada exatamente como se fosse um vetor.

A alocação utiliza a função **calloc()** para alocar um bloco de memória de tamanho suficiente para conter um vetor com **count** elementos de tamanho **eltSize** cada um. Veja!

```
c
#include <stdlib.h >
void * calloc (size_t count, size_t eltSize)
```

Atividade 2

Questão

Sabendo que um ponteiro é uma variável que armazena um endereço de memória, e a memória de um programa em execução é dividida em segmentos lógicos, como Text, Data, Stack e Heap, qual das seguintes afirmações é verdadeira sobre a utilização de ponteiros e a alocação de memória?

A

Ponteiros podem armazenar apenas endereços de variáveis globais, que estão alocadas no segmento Text.

B

A alocação dinâmica de memória utiliza o segmento Stack, onde são armazenados os parâmetros e as variáveis locais das funções.

C

O segmento Data é utilizado para armazenar o código do programa e suas constantes, enquanto o segmento Text armazena variáveis globais e estáticas.

D

Ponteiros podem ser utilizados para acessar memória alocada dinamicamente, que é gerenciada no segmento Heap.

E

O segmento Stack possui tamanho fixo e não varia durante a execução do programa.



A alternativa D está correta.

A alocação dinâmica de memória é feita no segmento Heap, que varia de tamanho durante a execução do programa. Ponteiros são utilizados para acessar essa memória alocada dinamicamente, permitindo maior flexibilidade no gerenciamento de recursos. As opções A, B, C e E estão incorretas. Ponteiros podem armazenar endereços de qualquer tipo de variável, não apenas variáveis globais (opção A). A alocação dinâmica de memória utiliza o Heap, não o Stack (opção B). O segmento Text armazena o código do programa, enquanto o segmento Data armazena variáveis globais e estáticas (opção C). O segmento Stack pode variar de tamanho durante a execução do programa (opção E).

Variáveis do tipo ponteiro

Os ponteiros são variáveis especiais em C que armazenam endereços de memória, permitindo acesso indireto a outras variáveis. Cada variável tem um nome, tipo, valor e endereço. Ponteiros possibilitam operações como leitura e alteração de valores sem utilizar diretamente o nome da variável.

Declarações de ponteiros utilizam o símbolo * e os operadores unários &; e * são usados para obter endereços e acessar valores. Já a capacidade de manipular endereços permite a criação de ponteiros para ponteiros, facilitando a manipulação complexa de dados. Esses ponteiros são essenciais para a alocação dinâmica de memória, a manipulação de arrays e a estruturação de dados, como listas, pilhas e árvores.

Assista ao vídeo para conferir as características específicas do ponteiro, e aprenda a aplicá-lo corretamente.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Ponteiro

Na linguagem C, cada variável tem um nome, um tipo, um valor e um endereço. Observe as variáveis a seguir.

```
c  
  
int x = 5;  
char c = 'D';
```

Nesse caso, o nome da variável é **x**, o tipo é inteiro, o valor é 5 e essa variável está armazenada na memória no endereço 10. Essa variável usa dois bytes de memória e quando um objeto usa mais de um byte, seu endereço é onde ele começa, no caso, 10.

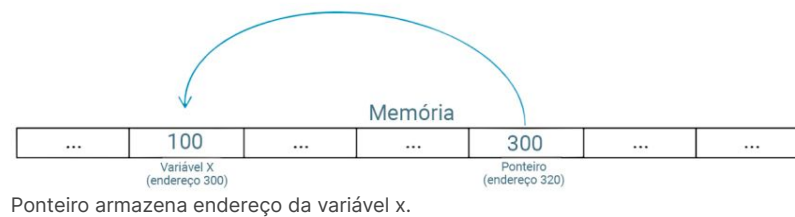
Assim como o nome da outra variável é **c**, o tipo char, o valor **D**, está armazenada no endereço de memória 13 e usa um byte de memória.

Na imagem a seguir, podemos observar a alocação dos valores das variáveis **x** e **c** na memória.



Já sabemos que todo dado armazenado em memória possui um endereço e que a definição de um ponteiro é uma variável que guarda o endereço de memória, ou seja, a localização do dado. Sendo assim, um ponteiro armazena o endereço de outra variável, isto é, uma variável que aponta para outra.

A imagem seguinte apresenta uma memória que armazena a variável **x** e o ponteiro para a variável também **x**. Como o conteúdo do ponteiro é um endereço, a seta indica a relação entre o ponteiro e a variável que ele aponta.



No exemplo, o valor da variável é 100 e esse dado está armazenado no endereço de memória 300. Como o ponteiro também ocupa espaço, o endereço de memória da variável **x** está armazenado no endereço 320 da memória, que é a posição de memória alocada para o ponteiro.

Uma das características importantes de um ponteiro é que, ao utilizá-lo, podemos realizar o acesso indireto a outras variáveis, isto é, podemos ler ou alterar o conteúdo de uma variável sem utilizar o nome dessa variável.

Declaração

Os ponteiros são declarados pelo símbolo ***** entre o tipo e o nome da variável. A forma geral da declaração é:

```
Tipo_da_variável * Nome_da_Variável;
```

Veja os exemplos a seguir, sabendo que as variáveis **p**, **q** e **r** são apontadores (ponteiro) para um inteiro, float e caractere, respectivamente.

- `int *p;`
- `float *q;`
- `char *r;`

Operadores

Agora confira dois tipos de operadores!

Operador unário & ou ponteiro constante

Tem a função de obter o endereço de memória de uma variável.

Operador unário * de indireção

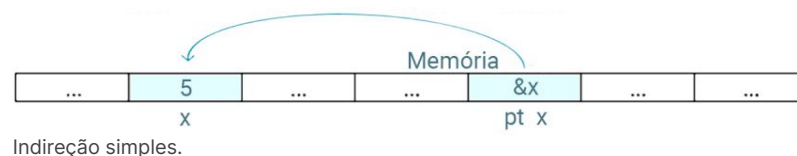
É usado para fazer a deferência.

Para entendermos melhor, observe o **Exemplo 2** a seguir. Nesse trecho de código, temos a definição de uma variável inteira e de um ponteiro para o tipo de dados inteiro.

```
c
int x = 5;
int *pt_x;

/* Ponteiro pt_x recebe o endereço de memória da variável x */
pt_x = &x;
```

A imagem a seguir mostra a indireção do exemplo.



O ponteiro **pt_x** recebe o endereço de memória da variável **x** pela instrução:

```
pt_x = &x;
```

Agora, o ponteiro **pt_x** terá armazenado o endereço de memória da variável **x**. Portanto, ao manusear o ponteiro **pt_x**, estará manipulando, indiretamente, a variável **x**.

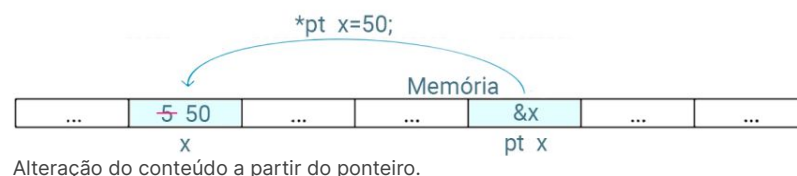
Para acessar o conteúdo do endereço armazenado no ponteiro, basta utilizar o operador ***** antes do nome do ponteiro. Veja!

```
*pt_x
```

Agora, considere a instrução apresentada a seguir. Note que o valor da variável **x** é alterado pelo ponteiro **pt_x**.

```
*pt_x = 50;
```

Podemos observar na imagem a seguir que o conteúdo da variável **x** foi alterado de forma indireta, ou seja, não foi feita a referência ao nome da variável **x**. Nesse caso, o ponteiro é chamado de ponteiro variável, pois assim é possível armazenar qualquer endereço.



Indireção múltipla

O endereço do ponteiro pode ser obtido da seguinte forma:

&pt_x

Assim, um ponteiro pode armazenar o endereço de outro ponteiro, ocasionando uma indireção múltipla.

Para entendermos melhor, vamos analisar a declaração de um ponteiro de indireção múltipla. Utiliza-se N vezes o operador *, sendo N o nível de indireção. No **Exemplo 3**, a seguir, é declarado um ponteiro para ponteiro.

```
c
int ** pt2; /*ponteiro para ponteiro do tipo inteiro*/
int * pt1; /*ponteiro para o tipo inteiro*/
int x = 10;

pt2 = &pt1;
pt1 = &x;

*pt1 = 30;
**pt2 = 50;
```

Sendo:

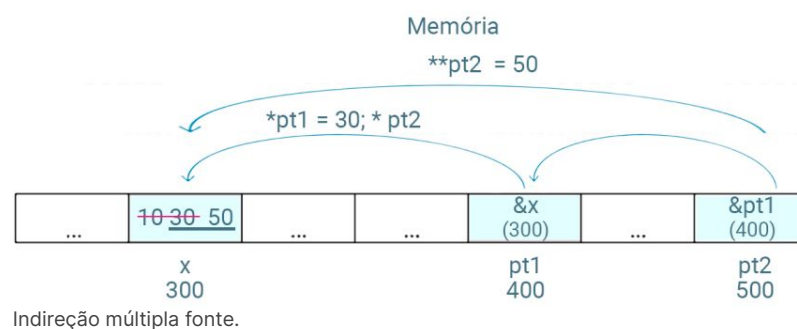
- &pt2: Endereço do ponteiro 'pt2';
- &pt1: Endereço do ponteiro 'pt1';

pt2: Conteúdo de 'pt2', ou seja, o endereço de 'pt1', que foi recebido através do comando pt2 = &pt1

pt1: Conteúdo de 'pt1', ou seja, o endereço de 'x', que foi recebido através do comando pt1 = &x

- *pt2: Conteúdo do endereço apontado, ou seja, o conteúdo de 'pt1'.
- *pt1: Conteúdo do endereço apontado, ou seja, o conteúdo de 'x'.
- **pt2: Acessa o conteúdo do endereço armazenado no ponteiro que é referenciado por 'pt2', ou seja, acessa 'x' indiretamente.

A imagem seguinte ilustra as operações com ponteiro. Veja!

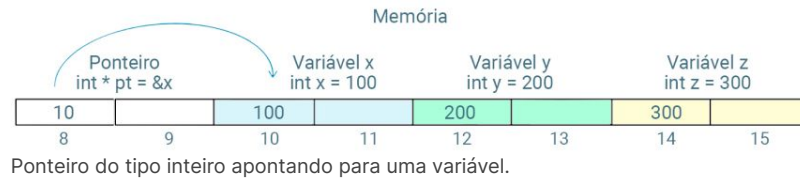


Aritmética de ponteiros

Os ponteiros permitem apenas as operações de adição e subtração. As operações de soma, subtração e comparação são válidas desde que os ponteiros envolvidos apontem para o mesmo tipo de dado.

No caso das operações básicas de incremento e decremento, analisaremos o comportamento e o que ocorre com o endereço armazenado no ponteiro quando uma das operações aritméticas é utilizada.

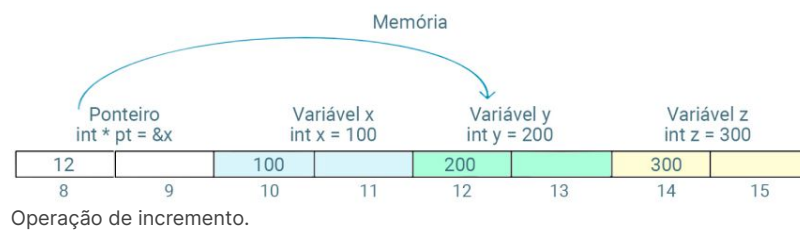
Na próxima imagem, podemos observar um ponteiro e três variáveis do tipo inteiro.



Consideremos que essas variáveis estão alinhadas sequencialmente na memória e que o ponteiro (pt) está armazenando o endereço da variável x (endereço 10).

Para incrementar o ponteiro: pt++;

Ao realizar a operação, o ponteiro passará a apontar para o próximo dado do seu tipo, ou seja, ao incrementar o ponteiro, o seu valor é alterado conforme o tamanho do tipo de dado que ele aponta. Diante disso, o próximo valor pt é o endereço 12, logo, o ponteiro aponta para a variável y. Essa situação é ilustrada na imagem a seguir.



Para decrementar o ponteiro: pt- -;

Neste caso, o ponteiro passa a apontar para o elemento anterior, ou seja, ao decrementar o ponteiro (pt), ele irá apontar novamente para a variável x que ocupa a posição 10.

De maneira geral, qualquer operação aritmética de ponteiro será realizada conforme o seu tipo base.



Exemplo

```
/* 4 x sizeof(int) = 16 bytes */ pt = pt + 4;
```

Como o ponteiro é do tipo inteiro e, para esse caso a variável inteiro ocupa 4 bytes, essa operação faz com que o ponteiro seja deslocado 16 bytes (4×4 bytes), ou seja, pt irá apontar para quatro elementos adiante.

Utilização dos ponteiros

Os ponteiros ou apontadores são muito úteis para acessar determinada variável em diferentes partes de um programa. Os ponteiros são úteis em várias situações, como:

- Alocação dinâmica de memória.
- Manipulação de arrays.
- Para retornar mais de um valor em uma função.
- Referência para listas, pilhas, árvores e grafos.

Exemplo de utilização de ponteiros

Vamos observar a seguir o **Exemplo 4** utilizando ponteiro.

c

```
#include < stdio.h >
#include < conio.h >
int main(void)
{
    //v_num é a variável que
    //será apontada pelo ponteiro
    int v_num = 10;

    //declaração de variável ponteiro
    int *ptr;

    //atribuindo o endereço da variável v_num ao ponteiro
    ptr = &v_num;

    printf("Utilizando ponteiros\n\n");
    printf("Conteúdo da variável v_num: %d\n", v_num);
    printf("Endereço da variável v_num: %x \n", &v_num);
    printf("Conteúdo da variável ponteiro ptr: %x", ptr);

    getch();
    return(0);
}
```

A saída da execução do Exemplo 4 é a seguinte:



Saída da execução do Exemplo 4.

O Exemplo 4 declara duas variáveis, a variável **v_num** do tipo **int** e a variável **ptr** do tipo **int** sendo um ponteiro, de acordo com as linhas 7 e 10 do código-fonte.

c

```
//v_num é a variável que será apontada pelo ponteiro
int v_num = 10;

//declaração de variável ponteiro
int *ptr;
```

Note que foi usado o operador ***** para designar que a variável **ptr** é um ponteiro. Como a intenção é armazenar o endereço da variável denominada **v_num**, que é uma variável do tipo **int**, o ponteiro também tem que ser do tipo **int**. Isso significa que vai apontar para uma variável do tipo inteiro.

Na linha 13, a variável **ptr** recebe o endereço de memória da variável **v_num**. Veja!

```
c
//atribuindo o endereço de v_num ao ponteiro
ptr = &v_num;
```

Para atribuir o endereço da variável `v_num` ao ponteiro `ptr`, utilizamos o operador de endereço `&`, pois estamos nos referindo ao endereço da variável `v_num` e não ao conteúdo.

Nas linhas 15 a 18, será impresso o valor da variável `v_num`, o endereço de memória da variável `v_num` e o valor do ponteiro `ptr`.

```
c
printf("Utilizando ponteiros\n\n");
printf("Conteúdo da variavel v_num: %d\n", v_num);
printf("Endereço da variavel v_num: %x \n", &v_num);
printf("Conteúdo da variavel ponteiro ptr: %x", ptr);
```

Foi utilizado `%x` para exibir o endereço e o conteúdo do ponteiro `ptr`, pois trata-se de um valor hexadecimal por ser endereço de memória.

Atividade 3

Questão

Considere a seguinte situação em um programa escrito em C:

Você deseja armazenar o endereço de uma variável em outra variável para manipular seu valor de forma indireta. Sabendo disso, a operação pode ser realizada ao utilizar ponteiros

A

declarando uma variável ponteiro com o operador `&` e atribuindo a ela o valor da variável.

B

declarando uma variável ponteiro com o operador `*` e atribuindo a ela o endereço da variável utilizando o operador `&`.

C

declarando uma variável ponteiro com o operador `&` e atribuindo a ela o valor hexadecimal da variável.

D

declarando uma variável ponteiro com o operador `*` e atribuindo a ela diretamente o valor da variável.

E

declarando uma variável ponteiro com o operador `*` e atribuindo a ela o endereço da variável utilizando o operador `*`.



A alternativa B está correta.

Para manipular indiretamente o valor de uma variável em C, você deve usar ponteiros. Primeiro, declara-se uma variável ponteiro utilizando o operador *. Em seguida, atribui-se a esse ponteiro o endereço da variável que se deseja manipular, utilizando o operador &. As outras opções estão incorretas. A opção A confunde a declaração de um ponteiro com a obtenção do endereço de uma variável. A opção C não está correta, pois não se atribui diretamente um valor hexadecimal ao ponteiro. A opção D incorretamente atribui diretamente o valor da variável ao ponteiro em vez de seu endereço. A opção E incorretamente utiliza o operador * para obter o endereço, quando deve ser o operador &.

Algoritmos e estruturas de dados

Estrutura de dados é um ramo da computação que estuda mecanismos para organizar e armazenar dados de forma eficiente, permitindo que algoritmos os manipulem de maneira otimizada. Algoritmos são conjuntos de passos usados para realizar tarefas específicas e, quando aplicados a dados organizados em estruturas adequadas, resultam em programas mais eficientes e eficazes.

Portanto, a escolha correta das estruturas de dados e dos algoritmos é essencial para o desenvolvimento de software que precisa processar grandes volumes de dados.

Entenda neste vídeo como se dá a estrutura de dados e observe que ela sempre deve estar associada a um algoritmo que a manipula.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Estrutura de dados

É o ramo da computação que estuda os diversos mecanismos de organização de dados para atender aos diferentes requisitos de processamento.



Resumindo

A estrutura de dados é um modo de armazenar e organizar os dados no computador, de uma maneira que eles possam ser usados eficientemente.

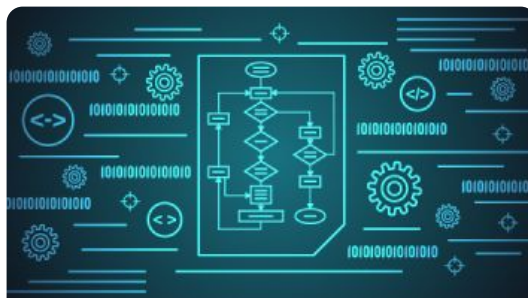
As estruturas de dados definem a organização, os métodos de acesso e as opções de processamento para coleções de itens de informação manipulados pelo programa. As estruturas são formas de distribuir e relacionar os dados disponíveis, de modo a tornar mais eficientes os algoritmos que manipulam esses dados.

Mas o que é algoritmo?

Algoritmo é um conjunto de passos finitos e organizados, que são usados para executar uma tarefa específica. Ele funciona como um bloco de informações, permitindo que dispositivos como computadores funcionem.

Já o programa é um código feito de algoritmos que dizem o que deve ser realizado.

Os algoritmos manipulam os dados. Quando os dados estão dispostos de forma coerente, caracterizam uma forma, uma estrutura de dados. Geralmente, os algoritmos são elaborados para manipulação de dados e, quando os dados estão organizados de forma coerente, representam uma estrutura de dados.



Processamento de algoritmos.



Comentário

Quando um programador cria o algoritmo para solucionar um problema, ele também cria uma estrutura de dados que é manipulada pelo algoritmo.

Atividade 1

Questão 1

Considere um programa de computador que necessita manipular uma grande quantidade de dados de maneira eficiente. Com base nessa situação, qual das seguintes afirmações descreve corretamente a relação entre estruturas de dados e algoritmos?

A

Estruturas de dados são responsáveis por definir os passos finitos e organizados de um algoritmo.

B

Algoritmos determinam a forma como os dados serão armazenados e organizados em estruturas de dados.

C

Estruturas de dados definem a organização e os métodos de acesso aos dados, enquanto algoritmos manipulam esses dados de forma eficiente.

D

Algoritmos são blocos de informações que dizem aos dados como devem ser organizados nas estruturas de dados.

E

Estruturas de dados e algoritmos são independentes e não influenciam a eficiência um do outro.



A alternativa C está correta.

Estruturas de dados são formas de organizar e armazenar dados de maneira eficiente, definindo a organização, os métodos de acesso e as opções de processamento dos dados. Algoritmos, por outro lado, são conjuntos de passos finitos e organizados que manipulam esses dados. A escolha correta da estrutura de dados pode melhorar a eficiência dos algoritmos que a manipulam. As outras opções estão incorretas. A opção A inverte a relação entre estruturas de dados e algoritmos; a opção B sugere que algoritmos determinam a organização dos dados, o que não é preciso; a opção D confunde a função de algoritmos e estruturas de dados; e a opção E erroneamente afirma que a eficiência de algoritmos e estruturas de dados é independente.

Tipos de estrutura de dados

As estruturas de dados são fundamentais para a organização e manipulação eficiente de informações em um programa de computador. Elas podem ser homogêneas, com elementos do mesmo tipo, como vetores e matrizes, ou heterogêneas, com elementos de tipos diferentes, como registros. Tipos de dados básicos incluem inteiro, real, lógico e texto.

Estruturas como listas, filas, pilhas, árvores e grafos são utilizadas para diferentes propósitos, desde armazenamento sequencial até representação de relações complexas. A escolha adequada da estrutura de dados é fundamental para a eficiência dos algoritmos que manipulam esses dados.

Assista ao vídeo e entenda melhor as estruturas de dados e suas diferentes classificações.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Uma estrutura de dados pode ser dividida em **dado** e **estrutura**.

O dado é o elemento que possui valor agregado e que pode ser utilizado para solucionar problemas computacionais.

Os dados possuem diferentes tipos. Por padrão, são quatro os tipos específicos de dados. Veja quais são eles!

Inteiro

Representa valores numéricos negativos ou positivos sem casa decimal, ou seja, valores inteiros.

Real

Representa valores numéricos negativos ou positivos com casa decimal, ou seja, valores reais. Também são chamados de ponto flutuante.

Lógico

Representa valores booleanos, assumindo apenas dois estados, verdadeiro ou falso. Pode ser representado apenas um bit (que aceita apenas 1 ou 0).

Texto

Representa uma sequência de um ou mais caracteres, colocamos os valores do tipo texto entre "" (aspa duplas).

O tipo de dados texto pode ser subdividido em string ou caractere, e o tipo de dado real em ponto flutuante com precisão simples (float) ou ponto flutuante com precisão dupla (double).

A estrutura é o elemento responsável por carregar as informações dentro de uma estrutura de software.

Vejamos alguns tipos de estrutura!

- Vetores multidimensionais
- Pilhas
- Filas
- Listas
- Árvores
- Grafos
- Tabelas hashing

A seguir, vamos conhecer um pouco mais dos principais tipos de estrutura de dados citados anteriormente.

1

Vetores unidimensionais e bidimensionais

Também conhecidos por arrays, são algumas das estruturas de dados mais simples e mais utilizadas. Os dados armazenados nesse tipo de estrutura são do mesmo tipo, portanto, dizemos que é uma estrutura de dados homogênea. Nessa estrutura, a adição e pesquisa de novos elementos é realizada de forma aleatória e o acesso aos elementos é através de índices. No caso dos vetores unidimensionais, só existe uma linha, e no caso dos bidimensionais, são empregadas linhas e colunas, formando uma matriz.

2

Lista encadeada

É uma estrutura de dados que implementa um conjunto de dados ordenados e do mesmo tipo, ou seja, é uma sequência de zero ou mais elementos de determinado tipo. Uma diferença básica entre os vetores e a lista é que nela os elementos não precisam estar fisicamente em sequência, ou seja, eles podem ocupar posições de memória não sequenciais.

3

Fila

É uma estrutura de dados com a ideia de organizar elementos em fila. Basicamente, uma fila é uma lista que só vai para um lado, ou seja, novos elementos são inseridos no final da fila e os elementos que serão removidos estão no início da fila. É comum utilizar a expressão FIFO (first-in-first-out) para ilustrar o funcionamento da fila, que significa primeiro a entrar - primeiro a sair.

4

Pilha

É uma estrutura de dados amplamente utilizada e que implementa a ideia de pilha de elementos. Basicamente, é uma sequência de elementos dispostos “um em cima do outro”, mas com uma regra para entrada e saída dos elementos. O último elemento que chega é o primeiro que sai da estrutura. É comum utilizarmos a expressão LIFO (last-in-first-out) para designar a pilha, que significa último a entrar - primeiro a sair. Para que o acesso a um elemento da pilha ocorra, os demais acima devem ser removidos.

5Árvore

É uma estrutura de dados que caracteriza uma relação de hierarquia entre os elementos. Por exemplo, para se construir um organograma de uma empresa, visualizando o conjunto de pessoas que lá trabalham, tendo em conta sua função, a estrutura de dados árvore é que mais se aplica. Na estrutura da empresa, uma pessoa não pode pertencer a dois departamentos diferentes, cada departamento tem a sua própria diretoria.

6

Grafo

É uma estrutura de dados bastante genérica, que organiza vários elementos, estabelecendo relações entre eles, dois a dois. Por exemplo, ao se estabelecer um trajeto para percorrer todas as capitais do NE do Brasil, devemos utilizar um mapa que indique as rodovias existentes e estabeleça uma ordem para percorrer as cidades. Para esse caso, a estrutura de dados grafo é mais apropriada.

Estruturas de dados homogêneas X heterogêneas

As estruturas de dados também se dividem em homogêneas e heterogêneas. Vamos conhecê-las!

As **estruturas homogêneas** são conjuntos de dados formados pelo mesmo tipo de dados. Essa estrutura permite o agrupamento de várias informações ou valores dentro de uma mesma variável. Como exemplo, podemos citar os vetores e as matrizes.

Um vetor é um arranjo de elementos que armazenamos na memória principal, com posições contíguas de memória, todos com o mesmo nome. Sendo assim, podemos entender que os vetores são estruturas lineares e estáticas, isto é, são compostas por um número finito e predeterminado de valores.

Na imagem a seguir, é possível observarmos um vetor de notas com 9 posições, em que cada posição corresponde a um índice do vetor.

Sérgio Matemática 10,0	Lucas Português 8,5	Alana História 10,0	Júlia Geografia 9,5	...	Daisy Física 7,5
0	1	2	3		n

Exemplo de vetor.

Nas estruturas de dados homogêneas, a individualização de cada variável é feita através do uso de índices. No caso dos vetores que são matrizes de uma só dimensão, é necessário apenas 1 índice para acesso às variáveis.

Na imagem seguinte, temos um exemplo de matriz. As matrizes possuem mais de uma dimensão, isto é, necessitam de um índice para cada dimensão para acesso às variáveis. E assim como nos vetores, cada posição dentro de uma matriz possui um índice. Observe!

		Posição do livro				
		0	1	2	...	n-1
Prateleira	0	788	598	265	...	156
	1	145	258	369	...	196
	2	989	565	345	...	526
	⋮	⋮	⋮	⋮	⋮	⋮
	m-1	845	153	564	892	210

Exemplo de matriz.

Já as **estruturas heterogêneas** são conjuntos de dados formados por tipos de dados diferentes, como os registros. Para entendermos melhor, citemos como exemplo uma agenda telefônica, na qual teremos informações de vários tipos: Nome, telefone, endereço etc.

Uma das principais estruturas de dados heterogêneas é o registro.



Comentário

Um registro pode ser composto de vários campos, componentes ou elementos, e cada um deles pode ser de um tipo diferente. Chamaremos cada elemento de um registro de campo, e para cada campo, teremos que definir o seu tipo e o seu identificador. É possível definir quantos campos forem necessários.

Por exemplo, imaginemos uma ficha de cadastro com os seguintes componentes: matrícula, nome, data de nascimento, cargo e salário. Esses campos do registro, que são de tipos diferentes, são partes que especificam cada uma das informações, conforme mostra a Tabela 3.

Registro Funcionário	
Matrícula	Tipo Inteiro
Nome	Tipo Cadeia de Caracteres
Dt. Nascimento	Tipo Data
Cargo	Tipo Cadeia de Caracteres
Salário	Tipo Real

Tabela 3: Campos do registro funcionários.
Daisy Albuquerque.

A definição dos campos de um registro é similar à declaração de variáveis. Portanto, assim como na declaração de variáveis, podemos declarar campos do mesmo tipo, na mesma linha, separados por vírgulas.

A declaração de um registro apresenta a estrutura a seguir, em que cada campo funciona como uma variável primitiva, mas todos os campos estão agrupados no registro. Veja!

```
<nome_da_variavel>:registro
```

```
Inicio
```

```
<nome_campo_1>:<tipo_campo_1>
```

```
<nome_campo_2>:<tipo_campo_2>
```

```
...
```

```
<nome_campo_n>:<tipo_campo_n>
```

```
Fimregistro
```

Na sequência, temos o exemplo de declaração da estrutura de dados heterogênea (registro) para a ficha de cadastro dos funcionários de uma empresa.

```
c
var Ficha_Funcionario: registro
inicio
    matricula: inteiro
    nome: vetor[1..50] de caractere
    Dt_Nascimento: vetor[1..9] de caractere
    Cargo: vetor[1..30] de caractere
    Salario: real
fim
```

A variável `Ficha_Funcionario` agrupa as informações referentes a um funcionário da empresa. Essa estrutura constitui um recurso importante para organizar os dados utilizados por um programa, pois trata um grupo de valores como uma única variável.



Comentário

Para acessar esse tipo de variável, é necessário especificar o registro nominando o campo que se deseja utilizar. Ou seja, para manipularmos um registro, temos que normalmente fazê-lo campo a campo, como no caso dos elementos dos vetores. A única diferença é que podemos fazer atribuição direta entre variáveis registro, o que não pode ser feito entre vetores. Ao se fazer uma atribuição entre duas variáveis registro, automaticamente o conteúdo de todos os campos de uma variável, mesmo que alguns deles sejam vetores, é copiado para os campos da outra variável.

Assim, para que se tenha acesso a um campo de uma variável registro, seja para utilizar o valor desse campo ou alterar o seu conteúdo, faremos da seguinte forma:

```
varReg.campo
```

Voltando ao nosso exemplo, para acessar o campo matrícula, nome e cargo, devem-se realizar as seguintes operações:

```
leia(Ficha_Funcionario.matricula)
```

```
escreva(Ficha_Funcionario.nome)
```

```
Ficha_Funcionario.cargo ← "gerente"
```

Os registros são geralmente utilizados quando queremos trabalhar com tabelas heterogêneas ou quando estamos trabalhando com arquivos binários. No caso de se trabalhar com tabelas heterogêneas, criamos uma tabela heterogênea simples criando-se um vetor de registros.



Comentário

Tanto os vetores como os registros são usados com a finalidade de resolver problemas mais complexos. Ao se considerar que um vetor é um conjunto de elementos do mesmo tipo, é natural que, quando precisamos agrupar vários registros, por exemplo, várias Fichas de Funcionário, utilizemos vetores de registro.

Os vetores de registro são utilizados para armazenar conjuntos de elementos complexos. No caso do exemplo, no lugar de armazenar apenas os dados do Funcionário, um vetor de registro pode armazenar as informações cadastrais de todos os 50 funcionários da empresa.

c

Algoritmo exemplo

var funcionario: vetor [1..50] de registro

Inicio

matricula: inteiro

nome: caractere

dtNascimento: caractere

cargo: caractere

salario: real

fimregistro

Para cada um dos elementos que constituem um registro, é dado o nome do campo. Sendo assim, no exemplo anterior, temos os campos: matrícula, nome, data de nascimento, cargo e salário.

Ao analisarmos melhor o exemplo, podemos pensar nesse vetor de registros como sendo uma tabela de 50 linhas e 5 colunas, em que a primeira coluna seria para guardar valores de inteiro simples, a segunda, terceira e quarta colunas para guardar valores strings e a quinta coluna para guardar valores de inteiro longo, conforme é ilustrado na Tabela 4.

Matrícula	Nome	D. Nascimento	Cargo	Salário

Tabela 4: Exemplo da tabela de registro de funcionário.
Daisy Albuquerque.

Atividade 2

Questão

Em um sistema de gerenciamento de funcionários, o programador precisa armazenar informações variadas, como matrícula, nome, data de nascimento, cargo e salário. Considerando os conceitos de tipos de dados e estruturas de dados, qual das seguintes opções é a mais adequada para organizar esses dados de maneira eficiente?

A

Utilizar um vetor unidimensional de inteiros para armazenar todas as informações.

B

Criar uma estrutura de dados heterogênea (registro) que agrupe todos os campos necessários.

C

Utilizar uma lista encadeada em que cada nó contém um único tipo de dado.

D

Utilizar uma pilha para armazenar todos os dados dos funcionários

E

Criar um vetor de caracteres para armazenar todas as informações como strings.



A alternativa B está correta.

A estrutura de dados mais adequada para armazenar informações variadas de funcionários é o registro, que permite agrupar diferentes tipos de dados em uma única unidade. Um registro pode conter campos de tipos variados, como inteiros para matrícula e salário, e strings para nome, data de nascimento e cargo, permitindo um acesso organizado e eficiente a cada campo. As outras opções são menos adequadas. A opção A não acomoda diferentes tipos de dados; a opção C não é eficiente para o agrupamento de diferentes tipos de dados; a opção D não é apropriada para o acesso direto e organizado dos dados; e a opção E não separa adequadamente os diferentes tipos de dados e campos.

Declaração e definição de structs

Na linguagem C, as structs são usadas para definir estruturas de dados heterogêneas, permitindo a combinação de variáveis de tipos diferentes sob um único nome. Elas são especialmente úteis para criar registros que armazenam informações diversas, como dados de um livro ou endereço. A definição de uma struct envolve a declaração de seus membros, que podem ser de qualquer tipo básico ou agregado.

As structs podem ser manipuladas de várias formas, incluindo inicialização, atribuição e acesso aos membros, tornando-as uma ferramenta poderosa para a organização e o gerenciamento de dados complexos em programas C.

Assista ao vídeo para saber como representar estruturas de dados heterogêneas em C. Vamos lá!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Definição de struct em C

Na linguagem C, existem dois tipos de dados: os tipos básicos (int, char, float, double, void) e os tipos compostos homogêneos (arrays). No entanto, nem sempre esses tipos são suficientes para o programa e, por isso, a linguagem C nos permite a criação de outras estruturas de dados, a partir dos tipos básicos, como os registros.

Na linguagem C, os registros são representados com uma estrutura denominada struct, que é uma coleção de variáveis relacionadas, usando um nome comum.

As estruturas de dados em linguagem C podem conter variáveis de tipos diferentes de dados, ao contrário de um array, que só pode conter dados de um mesmo tipo.



Comentário

As estruturas (struct) são usadas para definir registros que são armazenados em arquivos e, com os ponteiros, simplificam a criação de estruturas de dados mais complexas, por exemplo, pilhas, listas ligadas, filas e árvores.

Atenção: as estruturas são um tipo de dado derivado, o que significa que são construídas com o uso de objetos de outros tipos existentes. Observe a imagem a seguir e confira um modelo de estrutura na linguagem C.

```
c
struct identificador {
    tipo variável; tipo variável;
    tipo variável;
    tipo variável;
};
```

Em resumo, struct é um conjunto de variáveis sob o mesmo nome, e definido pelo programador. E cada variável dentro de uma estrutura pode ser de um tipo diferente, ou melhor, não é obrigatório que sejam todas do mesmo tipo.

Confira na sequência um exemplo de definição de struct. A palavra-chave **struct** inicia o bloco de definição da estrutura. Logo após, vem o nome que desejamos atribuir à estrutura (tag da estrutura), e então são abertas chaves “{” para iniciar a seção do corpo da estrutura.

```
c
struct mystruct {
    char a;
    int b;
    float c;
};
```

Neste outro exemplo, é possível visualizar a definição de uma estrutura de nome endereço, contendo os membros rua, número, CEP e bairro.

```
c
struct endereco {
    char rua[50];
    char numero[5];
    char CEP[8];
    char bairro[30];
};
```

Declaração de structs em C

A declaração de estrutura de dados struct é realizada da seguinte forma:

```
c
struct nome_da_estrutura {
    tipo_campo1 nome_campo1;
    tipo_campo2 nome_campo2;
    ...
} variáveis_que_armazenam_a_estrutura;
```

Em que:

- Struct é a palavra-chave para estrutura.

- Tag da estrutura é a palavra nome_da_estrutura. A tag de estrutura é usada para declarar variáveis deste tipo de estrutura, sendo o que identifica esse tipo de estrutura.
- Membros ou campos são as variáveis dentro da estrutura.
- Variáveis_que_armazenam_a_estrutura é o nome da variável que irá armazenar os valores da struct.

No corpo da estrutura, são definidas as variáveis ou membros do tipo struct, cada uma com seu nome, sem repetições, e podem ser de qualquer tipo básico de dados (int, float, char etc.), ou ainda, tipos de dados agregados, como arrays ou até mesmo outras estruturas – desde que não contenha uma instância de si mesma. É comum chamar os membros de elementos ou campos.

Note que nenhuma variável é realmente declarada com esse comando – apenas definimos o formato dos dados da estrutura. Basicamente, um tipo complexo de variável. Se quisermos declarar uma variável, podemos fazê-lo ao final da estrutura (**variáveis_que_armazenam_a_estrutura;**) ou com as declarações de outras variáveis.

A declaração da estrutura de dados struct pode ser feita de três maneiras diferentes.

A primeira é a declaração do tipo struct realizada da mesma forma que declaramos variáveis de outros tipos quaisquer. Por exemplo, vamos declarar a variável x para que seja do tipo struct endereço com a declaração a seguir. Veja!

```
c
struct endereco x;
```

A segunda maneira é a declaração de uma ou mais variáveis durante a definição da própria estrutura. Por exemplo, podemos declarar três variáveis de nomes livro1, livro2 e livro3 a partir de uma estrutura de nome livro.

```
c
struct livro {
    char nome[30];
    char autor[50];
    int paginas;
    float preco;
} livro1, livro2, livro3;
```

A terceira maneira é para o caso de apenas uma variável do tipo estrutura ser necessária no programa. Sendo assim, não precisamos declarar um identificador, basta definir o nome da variável que será criada. Por exemplo, podemos criar uma variável estrutura chamada livro, da seguinte forma:

```
c
struct {
    char nome[30];
    char autor[50];
    int paginas;
    float preco;
} livro;
```

Na última forma de declaração, o nome para a tag de estrutura é opcional. Entretanto, se não for declarado um nome, essa forma de declaração será a única maneira de declarar as variáveis do tipo estrutura. Apesar de ser possível não dar um nome à estrutura, sempre forneça um, pois é uma prática recomendada, tudo bem?

Podem ser feitas diversas operações com as structs em linguagem C, como:

- Atribuição de variáveis da estrutura a variáveis da estrutura do mesmo tipo.
- Leitura do endereço de uma variável de estrutura (operador &).
- Acesso aos membros de uma variável de estrutura.
- Uso do operador sizeof para determinar o tamanho de uma variável de estrutura.

Na sequência, veremos algumas dessas operações.

Inicializando structs em C

A estrutura de dados struct em linguagem C possui uma inicialização parecida com os vetores e as matrizes.



Comentário

Uma das maneiras de inicializar uma struct é através de uma lista de inicialização, como um array. Agora, se o número de inicializadores na lista for menor que os membros na estrutura, os membros restantes serão automaticamente inicializados em zero ou Null, se o membro for um ponteiro.

Por exemplo, podemos inicializar a estrutura endereço usando a declaração a seguir. Observe que os elementos do array são passados aos membros da estrutura na ordem em que foram declarados.

```
c
struct endereco x = {"Av. das Américas", "4200", " 22640-102 ", "Barra da Tijuca"};
```

Outra forma é atribuindo uma variável estrutura do mesmo tipo, já inicializada.

Por último, atribuindo valores aos membros individuais da estrutura. Por exemplo, suponha que desejamos atribuir o valor “Av. das Américas” ao membro rua da estrutura endereco.

Podemos usar a declaração a seguir, referenciando o nome da variável estrutura, seguida por um ponto e pelo nome do membro (campo) que receberá a atribuição do dado.

```
endereco.rua = "Av. das Américas";
```

Atividade 1

Questão

Em um programa em C, um desenvolvedor precisa armazenar informações sobre livros, incluindo nome, autor, número de páginas e preço. Qual das seguintes opções descreve corretamente como usar uma struct para essa finalidade?

A

Declarar quatro variáveis separadas para cada informação e usar um array para armazená-las.

B

Usar uma struct para agrupar todas as informações do livro sob um único nome e declarar variáveis dessa struct.

C

Criar um vetor de inteiros para armazenar todas as informações do livro.

D

Usar um ponteiro para armazenar os valores de cada informação do livro.

E

Declarar uma struct dentro de uma função e utilizar variáveis globais para cada informação do livro.



A alternativa B está correta.

A utilização de struct permite agrupar diferentes tipos de dados sob um único nome, facilitando a organização e manipulação de informações relacionadas. No exemplo dado, a struct pode conter campos para nome, autor, número de páginas e preço, proporcionando uma forma eficiente e estruturada de armazenar e acessar todos os dados de um livro. As outras opções estão incorretas. A opção A não utiliza struct e mantém as informações separadas, o que dificulta a organização; a opção C é inadequada porque um vetor de inteiros não pode armazenar diferentes tipos de dados; a opção D sugere o uso de ponteiros sem a necessidade específica de agrupar dados heterogêneos; e a opção E envolve uma abordagem desnecessariamente complexa e ineficiente para o problema descrito.

Manipulando structs

Para acessar e manipular os membros de uma struct em C, utiliza-se o operador de ponto (.) e o operador de seta (→). O operador de ponto é usado para acessar diretamente os membros da estrutura, enquanto o operador de seta é usado quando se trabalha com ponteiros para estruturas. Atribuições, impressões e leitura de valores dos campos podem ser realizadas diretamente ou por meio de funções. Essas operações são essenciais para a manipulação eficiente de dados estruturados em programas C, permitindo a organização e gestão de informações complexas.

Veja neste vídeo como manipular os dados armazenados em variáveis do tipo struct. Essa manipulação possui algumas características que você deve conhecer para programar corretamente.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Acessando os membros da struct em C

Para acessar os membros de uma estrutura de dados struct, podem ser utilizados dois tipos de operadores:

- Operador de membro de estrutura . (operador de ponto ou de seleção direta).
- Operador de ponteiro de estrutura → (operador de seta).

Quando é declarada uma variável do tipo estrutura, acessamos um membro da estrutura usando o operador ponto, indicando o nome da estrutura, seguido por um ponto e pelo nome do campo que se quer acessar. Nesse caso, dizemos que a estrutura é diretamente referenciada.

Por exemplo, podemos acessar e visualizar o campo rua da estrutura x, que é uma variável da struct endereço,) usando a seguinte declaração:

```
printf("%s", x.rua);
```

Quando a struct for referenciada através de ponteiros, o emprego é semelhante ao operador ponto, sendo esse substituído pelo operador seta (→).

Por exemplo, suponha que um ponteiro xptr tenha sido declarado e aponte para a estrutura endereco, e que o endereço de uma estrutura x tenha sido atribuído ao ponteiro xpri. Para acessar o membro rua da estrutura endereco via operador de ponteiro, usamos a declaração a seguir:

```
printf("%s", x→rua);
```

Manipulando structs

As estruturas de dados structs podem ser manipuladas com o objetivo de acessar os campos, atribuir valores para os campos, imprimir os valores dos campos, entre outros.

Para atribuir valores aos campos da estrutura, você faz isso diretamente, e em qualquer parte do programa, conforme a seguir.

```
c
aluno_especial.codigo = 10;
strcpy(aluno_especial.nome, "Manoel");
aluno_especial.nota = 10.0;
```

Para atribuir um valor a uma string, é necessário utilizar a função strcpy (CPY = copiar; STR = string). A função copiará o que está dentro das aspas duplas para o campo STRING da estrutura.

É possível imprimir os valores dos campos da estrutura em qualquer parte do programa utilizando a função printf.

```
c
printf(" \n %d ", aluno_especial.codigo);
printf(" \n %s ", aluno_especial.nome);
printf(" \n %.2f ", aluno_especial.nota);
```

Entretanto, se for preciso imprimir todos os membros da estrutura de uma única vez, é recomendável criar uma função para isso. Acompanhe!

```
c
void imprimir(Aluno aluno_regular){
    printf(" \n %d ", aluno_especial.codigo);
    printf(" \n %s ", aluno_especial.nome);
    printf(" \n %.2f ", aluno_especial.nota);
}
```

Observe que a função de impressão é do tipo Void, mas tem um argumento, isto é, é definida uma variável do tipo da estrutura, significando que, ao chamar a função, você deverá passar como parâmetro a estrutura que está trabalhando.

A chamada fica da seguinte forma:

```
imprimir(aluno_especial);
```

Para obter dados do teclado, devemos utilizar a função scanf, como em outras situações para entrada de dados na linguagem C. Podemos obter dados do teclado em qualquer parte do programa, assim como também podemos definir uma função para realizar esse trabalho.

```
c
printf(" Digite o código do aluno especial: ");
scanf("%d%c", &aluno_especial.codigo);
printf(" Digite o nome do aluno especial: ");
scanf("%s%c", &aluno_especial.nome);
printf(" Digite a nota do aluno especial: ");
scanf("%f%c", &aluno_especial.nota);
```

Observe que na função cadastrar e imprimir é passado como parâmetro (por valor) a variável do tipo estrutura. Em algumas ocasiões, você precisará fazer dessa forma, em outras, talvez você não tenha que passar a estrutura como parâmetro.

```
c
void cadastrar(Aluno aluno_especial){
    printf(" Digite o código do aluno especial: ");
    scanf("%d%c", &aluno_especial.codigo);
    printf(" Digite o nome do aluno especial: ");
    scanf("%s%c", &aluno_especial.nome);
    printf(" Digite a nota do aluno especial: ");
    scanf("%f%c", &aluno_especial.nota);
}
```

Exemplos práticos

No **Exemplo 5**, a seguir, vamos criar uma struct para armazenar os dados do aluno.

c

```
/* Cria uma estrutura para armazenar dados de um aluno*/
#include < stdio.h >
#include < stdlib.h >

struct aluno {
    int v_nmat; //número da matrícula
    float v_nota[3]; //notas
    float v_media; //media
};

int main() {
    struct aluno Felipe; //declara uma variável do tipo struct
    Felipe.v_nmat = 120;
    Felipe.v_nota[0]=8.5;
    Felipe.v_nota[1]=7.2;
    Felipe.v_nota[2]=5.4;
    Felipe.v_media=(Felipe.v_nota[0]+ Felipe.v_nota[1]+ Felipe.v_nota[2])/3.0;
    printf("Matricula:%d\n", Felipe.v_nmat);
    printf("Media: %2.f\n", Felipe.v_media);
    system("pause");
    return(0);
}
```

Criada a struct para armazenamento de dados do aluno, podemos extrair o seguinte do Exemplo 5:

- É declarada a struct aluno, conforme a linha 5 do código-fonte. A struct aluno é composta pelos campos matrícula, nota e média, em que o nome do campo matrícula é v_nmat do tipo int, o nome do campo nota é v_nota do tipo float e o nome do campo média é v_media do tipo float. O campo nota é também um vetor de 3 posições, conforme sua declaração na linha 7.
- Na linha 12, é declarada a variável Felipe do tipo struct aluno. A struct aluno Felipe recebe os valores para os seus campos nas linhas 13 a 17. Na linha 13, o campo v_nmat recebe o valor de 120. Nas linhas 14, 15 e 16, o campo v_nota, que é um array, recebe os valores para as três posições do vetor.
- Na linha 17, para o último campo da struct aluno é calculada a média do aluno e armazenada.
- O exemplo finaliza com a impressão da matrícula (linha 18) e da média (linha 19) das notas do aluno Felipe.

Após a execução do programa, temos o retorno com o valor das variáveis ilustradas na imagem a seguir.



Saída da execução do Exemplo 5 da struct para armazenar os dados do aluno.

Vamos analisar mais um exemplo.

No caso do **Exemplo 6** a seguir, criamos a struct `ficha_de_aluno`. Depois de criar a struct, precisamos criar a variável que vai utilizá-la. Para tanto, criamos a variável `aluno`, que será do tipo `ficha_de_aluno`.

c

```
/* Ficha de Aluno */
#include < stdio.h >
#include < conio.h >
int main(void)
{
    /*Criando a struct */
    struct ficha_de_aluno
    {
        char nome[50];
        char disciplina[30];
        float nota_prova1;
        float nota_prova2;
    };

    /*Criando a variável aluno que será do
    tipo struct ficha_de_aluno */
    struct ficha_de_aluno aluno;

    printf("\n----- Cadastro de aluno ----- \n\n\n");
    printf("Nome do aluno .....: ");
    fflush(stdin);
    fgets(aluno.nome, 40, stdin);
    printf("Disciplina .....: ");
    fflush(stdin);
    fgets(aluno.disciplina, 40, stdin);
    printf("Informe a 1a. nota ...: ");
    scanf("%f", &aluno.nota_prova1);
    printf("Informe a 2a. nota ...: ");
    scanf("%f", &aluno.nota_prova2);
    printf("\n\n ----- Lendo os dados da struct ----- \n\n");
    printf("Nome .....: %s", aluno.nome);
    printf("Disciplina .....: %s", aluno.disciplina);
    printf("Nota da Prova 1 ...: %.2f\n", aluno.nota_prova1);
    printf("Nota da Prova 2 ...: %.2f\n", aluno.nota_prova2);
    getch();
    return(0);
}
```

No Exemplo 6 visto anteriormente, é declarada a struct `ficha_de_aluno` composta pelos seguintes campos: `nome`, `disciplina`, `nota_prova1` e `nota_prova2`. O campo `nome` é uma variável do tipo `char` e um vetor com 50 itens, o campo `disciplina` é uma variável do tipo `char` e um vetor com 30 itens e os campos `nota_prova1` e `nota_prova2` são variáveis do tipo `float`.

Ao continuar com o reconhecimento do código-fonte do exemplo, é observado o uso da instrução `fgets()` (linhas 22 e 25) com o objetivo de ler strings, no caso, o nome do aluno e a disciplina.

`fgets(variavel, tamanho da string, entrada)`

Nesse caso, a entrada é `stdin` (entrada padrão), pois estamos lendo do teclado, mas, em outro caso, a entrada também poderia ser um arquivo.

Continuando, a struct `ficha_de_aluno` `aluno` recebe os valores para os campos `nota_prova1` e `nota_prova2` nas linhas 27 e 29.

Após a execução do programa, temos o retorno com o valor das variáveis ilustradas na imagem a seguir.

```
----- Cadastro de aluno -----  
  
Nome .....: Alana  
Disciplina .....: Inteligência Artificial  
Nota da Prova 1 .: 8  
Nota da Prova 2 .: 9  
  
----- Lendo os dados da struct -----  
  
Nome .....: Alana  
Disciplina .....: Inteligência Artificial  
Nota da Prova 1 .: 8.00  
Nota da Prova 2 .: 9.00
```

Saída da execução do Exemplo 6 da struct ficha_de_aluno.

Atividade 2

Questão

Em um programa C, um desenvolvedor está trabalhando com uma struct chamada `aluno_especial`, que contém os campos `código`, `nome` e `nota`. Ele precisa acessar e manipular esses campos, tanto diretamente quanto usando um ponteiro para a estrutura. Qual das seguintes opções descreve corretamente como essas operações podem ser realizadas?

A

Para acessar diretamente o campo `codigo`, utiliza-se `aluno_especial->codigo`.

B

Para acessar um campo usando um ponteiro, utiliza-se `aluno_especial.nome`.

C

Para atribuir um valor ao campo `nome`, deve-se usar a função `strcpy(aluno_especial.nome, "Manoel")`.

D

Para imprimir o campo `nota` usando um ponteiro, utiliza-se `printf("%f", aluno_especial->nota)`.

E

Para ler o campo `codigo` do teclado, utiliza-se `scanf("%d", &aluno_especial.codigo)`.



A alternativa C está correta.

A função strcpy é utilizada para copiar uma string para o campo nome da struct, que é do tipo char[]. As outras opções estão incorretas. A opção A usa o operador de seta incorretamente para acesso direto, mas deveria usar o operador de ponto (.); a opção B usa o operador de ponto incorretamente para um ponteiro, mas deveria usar o operador de seta (→); a opção D está correta na forma, mas a questão pedia a manipulação correta dos campos com e sem ponteiros; e a opção E está correta, mas a opção C aborda a manipulação de strings de forma correta e necessária no contexto.

Manipulando dados de alunos (struct)

Vamos analisar a manipulação de dados usando structs através de um exemplo prático. Nessa prática, você vai entender como criar e manipular estruturas de dados (structs) em C para armazenar informações complexas de forma organizada.

Na prática a seguir, a struct chamada aluno armazena nome, idade, matrícula e curso de um aluno. A struct permite agrupar variáveis de diferentes tipos sob um único nome, facilitando o acesso e a manipulação dos dados. Através de operadores de ponto e funções de entrada e saída, é possível ler, atribuir e imprimir os valores dos campos dessa estrutura.

Acompanhe neste vídeo o passo a passo sobre como declarar e manipular os dados de um aluno hipotético usando struct e array de structs em C.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Cadastro de alunos com structs

Passo 1: Preparar o ambiente.

1. Abra seu editor de código preferido.
2. Crie um novo arquivo chamado exemplo_array_structs.c.

Passo 2: Incluir bibliotecas necessárias.

```
c
#include
#include
```

Passo 3: Definir a struct.

```
c
struct Aluno {
    char nome[50];
    int idade;
    int matricula;
    char curso[30];
};
```

Passo 4: Implementação do programa principal.

```

c

int main() {
    int i;
    struct Aluno alunos[3]; // Array de structs para armazenar 3 alunos

    // Cadastrando dados de alunos
    strcpy(alunos[0].nome, "Sergio Silva");
    alunos[0].idade = 16;
    alunos[0].matricula = 1001;
    strcpy(alunos[0].curso, "Matematica");

    strcpy(alunos[1].nome, "Julia Pereira");
    alunos[1].idade = 17;
    alunos[1].matricula = 1002;
    strcpy(alunos[1].curso, "Fisica");

    strcpy(alunos[2].nome, "Joao Souza");
    alunos[2].idade = 18;
    alunos[2].matricula = 1003;
    strcpy(alunos[2].curso, "Quimica");

    // Exibindo dados dos alunos cadastrados
    printf("=== Dados dos Alunos Cadastrados ===\n");
    for (i = 0; i < 3; i++) {
        printf("Aluno %d:\n", i + 1);
        printf("Nome: %s\n", alunos[i].nome);
        printf("Idade: %d\n", alunos[i].idade);
        printf("Matricula: %d\n", alunos[i].matricula);
        printf("Curso: %s\n\n", alunos[i].curso);
    }

    return 0;
}

```

Passo 5: Compilação e execução.

1. Compile o programa usando um compilador C, como o GCC.

Atividade 3

Questão

Um desenvolvedor está criando um programa em C para gerenciar informações de alunos, incluindo nome, disciplina e notas. Ele decidiu usar structs para organizar esses dados. Com base nos exemplos fornecidos, qual das seguintes opções descreve corretamente como acessar e manipular os dados de uma struct?

A

Para acessar o campo nome de uma struct, usa-se `aluno->nome` quando a struct é referenciada diretamente.

B

Para ler um valor do campo `nota_prova1` a partir do teclado, usa-se `scanf("%s", aluno.nota_prova1)`.

C

Para inicializar o campo disciplina de uma struct, utiliza-se `aluno.disciplina = "Matemática"`.

D

Para acessar e imprimir o campo nome de uma struct utilizando um ponteiro, utiliza-se `printf("%s", aluno_ptr->nome)`.

E

Para calcular a média das notas armazenadas em uma struct, usa-se `media = aluno.nota_prova1 + aluno.nota_prova2 / 2`.



A alternativa D está correta.

A opção D está correta, pois o operador de seta (`→`) é usado para acessar os membros de uma struct através de um ponteiro. As outras opções estão incorretas. A opção A usa o operador de seta incorretamente para uma referência direta, enquanto deveria usar o operador de ponto (`.`); a opção B usa `scanf` com um especificador de formato incorreto, enquanto deveria ser `%f` para float; a opção C tenta atribuir uma string diretamente a um campo `char[]`, o que não é permitido, enquanto o correto seria usar `strcpy`; e a opção E contém um erro de precedência de operadores que resultaria em um cálculo incorreto. O correto seria usar parênteses para garantir a ordem correta das operações.

Structs aninhadas

Uma estrutura de dados aninhada em C é uma estrutura que contém outras estruturas dentro de si, permitindo a organização de dados complexos de forma hierárquica. Esse tipo de estrutura pode ser declarada de duas maneiras: colocando uma estrutura dentro de outra ou referenciando uma estrutura previamente definida dentro de outra estrutura. As structs aninhadas facilitam o gerenciamento de dados relacionados, como departamentos e cargos dentro de uma estrutura de funcionários, permitindo um acesso organizado e eficiente a cada nível de dados.

Confira neste vídeo como aninhar structs, ou seja, usar structs dentro de structs, o que pode ser necessário para modelar algumas situações práticas.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Structs aninhadas

Definição

Uma estrutura de dados aninhada ou struct aninhada é basicamente uma estrutura dentro de outra, ou seja, uma estrutura contida em outra ou uma estrutura que pode ser acessada por outra.

A struct aninhada é um aninhamento de estruturas que nos permite criar estruturas que contenham outras estruturas internas.

O padrão ANSI C especifica que as estruturas de dados (struct) podem ser aninhadas até 15 níveis, mas a maioria dos compiladores permite mais.

Declaração

Antes de iniciarmos, vamos rever alguns pontos importantes para o entendimento da declaração das estruturas aninhadas.



Relembrando

Sabemos que a declaração de estrutura declara um tipo struct. E cada tipo struct recebe um nome (ou tag) que se refere àquele tipo pelo nome precedido pela palavra struct. Cada unidade de dados na estrutura é chamada membro e possui um nome de membro. Os membros de uma estrutura podem ser de qualquer tipo. Sendo assim, os membros de estruturas podem ser também outras estruturas.

Veremos, a seguir, que as estruturas aninhadas podem ser declaradas de duas formas.

A primeira forma é colocar uma estrutura dentro de outra, literalmente. Portanto, a sintaxe para trabalhar com estruturas aninhadas nessa forma é:

```
c
typedef struct {
    tipo membro_1;
    tipo membro_2;
    tipo membro_n;
    struct{
        tipo membro_interno_1;
        tipo membro_interno _2;
        tipo membro_interno _n;
    }
} Nome_estrutura;
Nome_estrutura NE;
```

E para acessar os membros, o acesso é direto e a sintaxe é:

```
c
//Para atribuir valores:
NE.membro_interno_1 = 0;
NE.membro_1 = 0;

//Para leitura do teclado:
&NE.membro_interno_1
&NE.membro_1

//Para impressão:
NE.membro_interno 1
NE.membro_1
```

A outra forma é você declarar a estrutura_1 antes e, na estrutura_2, declarar uma variável da estrutura 1. Dessa forma, a sintaxe é a seguinte:

```
c
typedef struct {
    tipo membro_1;
    tipo membro_2;
    ...
    tipo membro_n;
} nome_estrutura_1;

typedef struct {
    tipo membro_1;
    tipo membro_2;
    nome_estrutura_1 NE1;
    ...
    tipo membro_n;
} nome_estrutura_2;

nome_estrutura_2 NE2;
```

E para acessar os membros, a sintaxe é:

```
c
//para acessar os membros
NE2.membro1;
NE2.membro2;
NE2.NE1.membro1;
NE2.NE1.membro2;
```

Para entendermos melhor as duas formas de declaração de estrutura aninhadas, vamos analisar o **Exemplo 7** a seguir.

```
c

#include < stdio.h >
#include < stdlib.h >

struct departamento {
    int cod;
    char descricao[30];
};

struct cargo {
    int cod;
    char descricao[30];
};

struct funcionario {
    int cod;
    char nome[30];
    float salario;
    struct departamento depto;
    struct cargo cargo;
};

struct funcionario Funcionario;

int main(void)
{

}
```

Observe as linhas de 4 a 20. Nesse trecho, são definidas três estruturas que recebem os nomes: departamento, cargo e funcionário. A estrutura funcionário possui 5 membros, os dois últimos membros são do tipo struct departamento e struct cargo, isto é, dentro da estrutura funcionário, temos dois membros do tipo struct.

Em resumo, temos:

1. Nas linhas 4 e 7, foi declarada a estrutura departamento.
2. Nas linhas 9 e 12, foi declarada a estrutura cargo.
3. Nas linhas 14 e 20, foi declarada a estrutura funcionário.
4. Na linha 18, o campo depto é do tipo struct departamento.
5. Na linha 19, o campo cargo, da mesma forma que o campo depto, é do tipo struct cargo.

Ao analisar o código, visualizamos uma struct aninhada nas linhas 18 e 19, com os campos depto e cargo. Temos, então, um aninhamento de estruturas.

Outra forma de declarar uma estrutura é colocar uma estrutura dentro da outra, literalmente, como apresentado no **Exemplo 8** a seguir.

c

```
#include < stdio.h >
#include < stdlib.h >

struct cargo {
    int cod;
    char descricao[30];
};

struct funcionario {
    int cod;
    char nome[30];
    float salario;
    struct departamento {
        int cod;
        char descricao[30];
    };

    struct cargo cargo;
};

struct funcionario Funcionario;

int main(void)
{

}
```

Além das duas formas de declaração de estruturas aninhadas mostradas anteriormente, pode-se utilizar também o tipo typedef ao invés de struct.

Sendo assim, vamos analisar, a seguir, o **Exemplo 8 adaptado** para o uso de typedef.

c

```
#include < stdio.h >
#include < stdlib.h >

typedef struct departamento {
    int cod;
    char descricao[30];
} Departamento;

typedef struct cargo {
    int cod;
    char descricao[30];
} Cargo;

typedef struct funcionario {
    int cod;
    char nome[30];
    float salario;
    Departamento depto;
    Cargo cargo;
} Funcionario;
printf( "\n TEXTO %_" , nome_vetor_struct[indice].nome_membro_struct);

int main(void)
{

}
```

Ao analisar o código, podemos observar que o typedef foi declarado em conjunto com o struct, tanto para a estrutura departamento quanto para o cargo e para a estrutura funcionário. Isso é útil apenas para resumir a codificação, mas não influi no desempenho da aplicação.



Comentário

O comando typedef permite criar um novo nome para outro tipo de dados. Assim, o novo tipo de dados pode ser declarado como um tipo de dados primitivo existente no Linux.

Ao analisar a declaração dos campos depto e cargo, observe que esses campos agora não são mais do tipo estrutura, mas dos tipos definidos **Departamento** e **Cargo**, respectivamente.

Dessa forma, pode-se declarar campos de outros tipos definidos dentro de uma estrutura que também pode ser um tipo definido. Além disso, não há uma limitação para o número de níveis na declaração aninhada, contudo, não seria prático incluir muitos níveis, pois isso irá complicar o entendimento para manutenção do código-fonte.

Manipulação de structs aninhadas

Agora que já sabemos como declarar uma estrutura aninhada, vamos aprender como manipular os membros de uma struct aninhada.

Observe o trecho do programa que define duas estruturas, porém a segunda tendo membros que são também estruturas.

c

```
#define LEN 50

struct endereco {
    char rua[LEN];
    char cidade_estado_cep[LEN];
};

struct student {
    char id[10];
    int idade;
    struct endereco casa;
    struct endereco escola;
};

struct student pessoa;
```

Dadas estas definições, é possível acessar os seguintes campos da variável pessoa do tipo struct (uma variável do tipo struct student):

c

```
pessoa.id
pessoa.casa.rua
pessoa.casa.cidade_estado_cep
pessoa.escola.rua
pessoa.escola.cidade_estado_cep
```

Note o uso repetido de “.” quando se acessa membros dentro de membros.

Atividade 1

Questão

Um programador está desenvolvendo um sistema de gerenciamento de funcionários e precisa armazenar informações sobre o departamento e o cargo de cada funcionário. Ele decide utilizar structs aninhadas para organizar esses dados. Com base nos conceitos de structs aninhadas, qual das seguintes opções descreve corretamente como acessar e manipular esses dados?

A

Para acessar o campo cod da estrutura departamento dentro da estrutura funcionario, usa-se funcionario.depto→cod.

B

Para acessar o campo descricao da estrutura cargo dentro da estrutura funcionario, usa-se funcionario->cargo.descricao.

C

Para definir uma struct aninhada, deve-se usar sempre o operador typedef.

D

Para acessar o campo cod da estrutura departamento dentro da estrutura funcionario, usa-se funcionario.depto.cod.

E

Para declarar uma struct aninhada, é necessário declarar todas as structs dentro de uma única definição.



A alternativa D está correta.

A opção D está correta porque o operador de ponto (.) é usado para acessar membros de uma estrutura, incluindo membros de estruturas aninhadas. As outras opções estão incorretas. A opção A usa o operador de seta (→) incorretamente, pois ele é usado apenas com ponteiros. A opção B também usa o operador de seta (→) incorretamente para acesso direto. A opção C sugere que o uso de typedef é obrigatório, o que não é verdade; é opcional. A opção E está incorreta porque structs aninhadas podem ser definidas separadamente e referenciadas dentro de outras structs.

Praticando struct aninhada

Struct aninhada em C permite a criação de estruturas de dados complexas e organizadas, facilitando a manipulação de dados relacionados. Esta prática ilustra como structs aninhadas podem ser implementadas e acessadas em diferentes contextos, proporcionando flexibilidade e clareza no gerenciamento de dados.

Agora é sua vez de aplicar a struct aninhada!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Passo 1: Preparar o Ambiente.

1. Abra seu editor de código preferido.
2. Crie um novo arquivo chamado exemplo_struct_aninhada.c.

Passo 2: Incluir bibliotecas necessárias.

```
c
#include
#include
```

Passo 3: Definir as structs.

```
c
struct Disciplina {
    char nome[30];
    char professor[50];
    int cargaHoraria;
};

struct Aluno {
    char nome[50];
    int idade;
    int matricula;
    struct Disciplina disciplinas[3];
};
```

Passo 4: Definição da função utilizada para o cadastro de disciplina.

```
c
void cadastrarDisciplina(struct Disciplina *disciplina, const char *nome, const char
*professor, int cargaHoraria) {
    strcpy(disciplina->nome, nome);
    strcpy(disciplina->professor, professor);
    disciplina->cargaHoraria = cargaHoraria;
}
```

Passo 5: Definição da função utilizada para o cadastro de aluno.

```
c
void cadastrarAluno(struct Aluno *aluno, const char *nome, int idade, int matricula) {
    strcpy(aluno->nome, nome);
    aluno->idade = idade;
    aluno->matricula = matricula;

    cadastrarDisciplina(&aluno->disciplinas[0], "Matematica", "Prof. Joao Silva", 60);
    cadastrarDisciplina(&aluno->disciplinas[1], "Historia", "Prof. Julia Costa", 45);
    cadastrarDisciplina(&aluno->disciplinas[2], "Quimica", "Prof. Carlos Souza", 50);
}
```

Passo 6: Definição da função utilizada para a exibição de alunos cadastrados.

```

c

void exibirAluno(struct Aluno *aluno) {
    int i;

    printf("Nome do Aluno: %s\n", aluno->nome);
    printf("Idade: %d\n", aluno->idade);
    printf("Matricula: %d\n", aluno->matricula);

    printf("Disciplinas:\n");
    for (i = 0; i < 3; i++) {
        printf("  Disciplina %d:\n", i + 1);
        printf("    Nome: %s\n", aluno->disciplinas[i].nome);
        printf("    Professor: %s\n", aluno->disciplinas[i].professor);
        printf("    Carga Horaria: %d horas\n", aluno->disciplinas[i].cargaHoraria);
    }
}

```

Passo 7: Implementação do programa principal.

```

c

int main() {
    struct Aluno aluno1;
    cadastrarAluno(&aluno1, "Alana Lima", 20, 1234);
    exibirAluno(&aluno1);
    return 0;
}

```

Passo 8: Compilação e execução.

1. Compile o programa usando um compilador C, como o GCC.

Atividade 2

Questão

Um desenvolvedor está criando um programa em C para gerenciar informações de alunos, incluindo código, nome e data de nascimento. Ele decidiu usar structs aninhadas para organizar esses dados. Qual das seguintes opções descreve corretamente como declarar e acessar essas informações utilizando structs aninhadas?

A

Declarar todos os campos diretamente dentro da struct Aluno e acessar usando aluno.dia.

B

Declarar a struct Data dentro da struct Aluno e acessar os membros usando aluno.Data.dia.

C

Declarar a struct Data separadamente e referenciá-la dentro da struct Aluno, acessando os membros usando aluno.datNasc.dia.

D

Declarar todos os campos dentro da struct Aluno e acessar usando aluno→dia.

E

Declarar a struct Data e Aluno dentro de uma única struct e acessar usando aluno.Data.dia.



A alternativa C está correta.

A opção C está correta porque, ao declarar a struct Data separadamente e referenciá-la dentro da struct Aluno, é possível acessar os membros da struct aninhada usando a sintaxe aluno.datNasc.dia. As outras opções estão incorretas porque não seguem a convenção correta para a declaração e o acesso de structs aninhadas em C.

Array de structs em C

São estruturas de dados que permitem armazenar e organizar uma sequência de objetos do mesmo tipo, em que cada objeto é uma estrutura que pode conter múltiplos campos de diferentes tipos. Esses arrays são particularmente úteis para criar listas de registros, como dados de alunos, em que cada posição do array pode armazenar informações como nome, disciplina e nota.

Através de operações de inicialização, inserção, impressão e busca, é possível gerenciar eficientemente grandes conjuntos de dados estruturados, facilitando a implementação de programas que requerem manipulação de registros complexos.

Assista ao vídeo e imagine que você tem uma turma composta por diversos estudantes, então, é razoável agregá-los de alguma forma. Vamos ver como fazer isso usando uma array!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Array de structs

Definição

Ao analisar o vetor da imagem a seguir, podemos verificar que cada posição do vetor armazena um conjunto de informações diferentes. Neste exemplo, cada posição está armazenando um tipo string no campo nome do aluno, um tipo de string no campo nome da disciplina e um tipo float no campo nota.

Sérgio Matemática 10,0	Lucas Português 8,5	Alana História 10,0	Júlia Geografia 9,5	...	Daisy Física 7,5
0	1	2	3		n

Exemplo de vetor de estrutura (array de structs).

Sendo assim, podemos conceituar um vetor de estruturas ou um array de structs como uma estrutura de dados que armazena uma sequência de objetos, que são do mesmo tipo, e no caso, os objetos são estruturas.



Comentário

Os arrays de structs são utilizados principalmente na criação de listas, podendo também ser usados em tarefas simples, como na repetição da leitura de dados de um cadastro.

Declaração

Para entendermos melhor a declaração de um array de structs, vamos voltar para a imagem do vetor de estrutura (array de structs) apresentada anteriormente.

A declaração da estrutura ficará da seguinte forma:

```
c

typedef struct {
    char nome[200];
    char disciplina[100];
    float nota;
} Aluno;

Aluno aluno_nota[10];
```

Ao observarmos a declaração da estrutura, a linha 7 é a que cria de fato o vetor. No exemplo, é criado um vetor de 10 posições para armazenar o nome do aluno, a disciplina que o aluno está cursando e a nota do aluno.

Inicializando o array de struct

Para inicializar um array de struct, devemos atribuir aos seus membros os valores padrão (default) de cada tipo de dado. O código a seguir mostra como isso é feito.

```
c

for(i=0; i< 10; i++) {
    strcpy(aluno_nota [i].nome, " NULL");
    strcpy(aluno_nota [i].disciplina, " ");
    aluno_nota [i].nota = 0.0;
}
```

Observe que a função STRCPY da biblioteca string foi utilizada. Essa função copia um valor para uma variável de cadeia de caracteres.

Populando o array de struct

Após a inicialização do array de struct, podemos inserir informações, dados. Sendo assim, é necessário um laço for para armazenar cada estrutura em uma posição.

O código a seguir ilustra como armazenar as informações do nosso funcionário. Dentro do for, colocamos os comandos de leitura e escrita, necessários para pedir ao usuário o que ele deve digitar e, depois, armazenar o que foi digitado. Note que, para armazenar os dados que são inseridos a partir do teclado, precisamos utilizar a seguinte sintaxe:

```
c
&nome_vetor_struct[indice].nome_membro_struct;

For (i=0; i< 10; i++) {
    printf(" Digite nome do aluno: ");
    scanf("%s%c", & aluno_nota [i].nome);
    printf(" Digite a disciplina do aluno: ");
    scanf("%s%c", & aluno_nota [i].disciplina);
    printf(" Digite o nota do aluno: ");
    scanf("%f%c", & aluno_nota [i].nota);
}
```

Imprimindo o array de struct

No caso da impressão dos valores do array de struct, será necessário o uso do laço for para percorrer todas as posições desse array. A cada passagem no for, será impresso o conteúdo de uma posição do vetor na tela. A sintaxe que você deve usar aqui é a seguinte:

```
printf( "\n TEXTO %_" , nome_vetor_struct[indice].nome_membro_struct);
```

É necessária a inclusão do tipo de dado que deverá ser impresso, isto é, substitua %_ pelo correspondente (exemplo: %d para inteiro).

```
c
for(i=0; i< 10; i++) {
    printf(" \n O nome do aluno é: %d ", aluno_nota [i].nome);
    printf(" \n A disciplina do aluno é: %d ", aluno_nota [i].disciplina);
    printf(" \n A nota do aluno é: %.2f ", aluno_nota [i].nota);
}
```

Buscando um elemento no array de struct

A pesquisa no array de struct pode procurar por um valor no vetor de estruturas. Por exemplo, você pode fazer uma pesquisa pelo nome, pela disciplina e pela nota, sendo assim, é possível fazer uma pesquisa por todos os membros da struct. Entretanto, é necessário tomar cuidado quando for buscar por uma string.

Vamos analisar o código de uma busca por uma string, nesse caso, vamos procurar pelo membro nome.

```

c

printf(" \n Digite um nome: ");
scanf("%s%c", nome);

for(i=0; i< 10; i++) {
    if(strcmp(nome, aluno_nota [i].nome)== 0) {
        printf("\n Registro encontrado! ");
        posicao = i;
    } else {
        posicao = -1;
    }
}

if(posicao = -1) {
    printf(" \n Registro não encontrado! ");
} else {
    printf(" \n Registro Encontrado: ");
    printf(" \n O nome do aluno é: %s ", aluno_nota [posicao].nome);
    printf(" \n A disciplina do aluno é: %s ", aluno_nota [posicao].disciplina);
    printf(" \n A nota do aluno é: %.2f ", aluno_nota [posicao].nota);
}

```

Agora, vamos analisar o código desse programa.

Primeiro, é solicitado ao usuário que digite um nome para a busca. Esse nome será armazenado em uma variável chamada nome, que foi declarada anteriormente no programa.



Comentário

Como o nome terá de ser procurado neste array, então um laço for será usado para percorrer esse array e analisar, posição a posição, se o nome digitado pelo usuário está ali ou não. Dessa forma, a string digitada pelo usuário será comparada com a string armazenada na posição i do vetor.

Uma função chamada strcmp(string1, string2); da biblioteca de strings é utilizada para fazer a comparação da primeira string com a segunda string. Aqui, a variável nome será comparada com o nome que está armazenado em cada uma das posições do vetor (aluno_nota [i].nome).

Quando o resultado dessa comparação for igual a zero, significa que a string foi encontrada em determinada posição do vetor. Mas se o resultado for diferente de zero, então a string não foi encontrada. Portanto, um if é usado dentro do for para fazer essa verificação.

```
if(strcmp(nome, aluno_nota [i].nome)== 0)
```

O if é adicionado logo depois do for para verificar se posição é -1 ou não. Se a posição for igual a -1, então o registro não foi encontrado, caso contrário, é impresso o valor da posição da estrutura.

Para que isso seja possível, a variável i, que seria o índice do vetor, é substituída pela variável posicao, que contém o valor exato da posição em que o nome foi encontrado.

Atividade 3

Questão 1

Você está desenvolvendo um sistema em C para gerenciar informações de alunos, incluindo nome, disciplina e nota. Para isso, decidiu utilizar arrays de structs para armazenar os dados de 10 alunos. Qual das opções a seguir descreve corretamente como declarar, inicializar e acessar os dados de um array de structs em C?

A

Declarar o array de structs como `struct aluno[10]` e acessar usando `aluno[i].campo`.

B

Declarar o array de structs como `struct Aluno aluno_nota[10]`, inicializar usando um loop for e acessar os campos com `aluno_nota[i].campo`.

C

Declarar o array de structs como `Aluno aluno_nota[10]`, inicializar diretamente nos campos e acessar usando `aluno[i].campo`.

D

Declarar o array de structs como `Aluno aluno_nota[10]`, inicializar cada campo fora do loop for e acessar usando `aluno_nota[i]→campo`.

E

Declarar o array de structs como `struct aluno_nota[10]` e acessar usando `aluno[j].campo`.



A alternativa B está correta.

A opção B está correta porque segue a convenção apropriada para declaração e inicialização de arrays de structs em C. A struct `Aluno` é definida, um array `aluno_nota` de 10 posições é declarado e inicializado usando um loop for. Os campos são acessados usando a sintaxe `aluno_nota[i].campo`, que é a maneira correta de acessar membros de structs dentro de um array. As outras opções não seguem corretamente todas as etapas necessárias ou utilizam sintaxe incorreta.

Manipulando dados de vários produtos

Em programação C, as estruturas (structs) são usadas para agrupar variáveis de diferentes tipos sob um único nome. A utilização de array de structs é particularmente útil na gestão de conjuntos de dados complexos, como listas de registros de produtos, em que cada elemento do array pode armazenar diversas informações relacionadas, como nome, código, quantidade e preço. Esse tipo de abordagem é essencial para criar programas que precisam manipular grandes volumes de dados organizados de maneira eficiente e estruturada.

Agora será sua vez de fazer a manipulação de structs agregadas por um array!



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Passo 1: Preparar o ambiente.

1. Abra seu editor de código preferido.
2. Crie um novo arquivo chamado exemplo_array_struct.c.

Passo 2: Incluir bibliotecas necessárias.

```
c  
  
#include  
#include
```

Passo 3: Definir uma variável com número fixo.

```
c  
  
#define NUM_PRODUTOS 5
```

Passo 4: Definir a struct.

```
c  
  
struct Produto {  
    char nome[50];  
    int codigo;  
    int quantidade;  
    float preco;  
};
```

Passo 5: Implementação do programa principal.

c

```
int main() {
    int i;
    struct Produto produtos[NUM_PRODUTOS];
    // Preenchendo os dados dos produtos
    strcpy(produtos[0].nome, "Camiseta");
    produtos[0].codigo = 101;
    produtos[0].quantidade = 50;
    produtos[0].preco = 29.99;

    strcpy(produtos[1].nome, "Calca Jeans");
    produtos[1].codigo = 102;
    produtos[1].quantidade = 30;
    produtos[1].preco = 79.99;

    strcpy(produtos[2].nome, "Tenis Esportivo");
    produtos[2].codigo = 103;
    produtos[2].quantidade = 20;
    produtos[2].preco = 199.99;

    strcpy(produtos[3].nome, "Bolsa de Couro");
    produtos[3].codigo = 104;
    produtos[3].quantidade = 15;
    produtos[3].preco = 149.99;

    strcpy(produtos[4].nome, "Oculos de Sol");
    produtos[4].codigo = 105;
    produtos[4].quantidade = 10;
    produtos[4].preco = 89.99;

    // Exibindo os dados dos produtos
    printf("=== Estoque de Produtos ===\n");
    for (i = 0; i < 5; i++) {
        printf("Produto %d:\n", i + 1);
        printf("  Nome: %s\n", produtos[i].nome);
        printf(" Codigo: %d\n", produtos[i].codigo);
        printf(" Quantidade: %d\n", produtos[i].quantidade);
        printf("  Preço: R$ %.2f\n\n", produtos[i].preco);
    }

    return 0;
}
```

Passo 6: Compilação e execução.

1. Compile o programa usando um compilador C, como o GCC.

Atividade 4

Questão

Você está desenvolvendo um sistema em C para gerenciar informações de alunos, incluindo código, nome e data de nascimento. Para simplificar o uso de structs, você decidiu usar a instrução typedef. Qual das opções a seguir descreve corretamente como declarar e inicializar um array de structs usando typedef?

A

Declarar o array de structs como typedef struct aluno Aluno[5] e acessar usando Aluno[i].campo.

B

Declarar o array de structs como struct Aluno aluno[5], inicializar usando um loop for e acessar os campos com aluno[i].campo.

C

Declarar a estrutura usando typedef struct { int codigo; char nome[200]; int dia; int mes; int ano; } Aluno; e criar o array com Aluno aluno[5];.

D

Declarar a estrutura usando typedef struct { int codigo; char nome[200]; Data datNasc; } Aluno; e inicializar o array com Aluno aluno[5]; e um loop for.

E

Declarar a estrutura como typedef struct { int codigo; char nome[200]; struct { int dia; int mes; int ano; } datNasc; } Aluno; e acessar os membros diretamente.



A alternativa D está correta.

A opção D está correta porque mostra a maneira adequada de usar typedef para definir uma estrutura e criar um array de structs. A estrutura Aluno contém um membro do tipo Data, que é outra struct. O array de structs é declarado e inicializado corretamente usando um loop.

Considerações finais

- Uso de ponteiros e manipulação de memória.
- Como alocar memória.
- Quais são os ponteiros variáveis.
- O que são estruturas de dados e sua relação com os algoritmos.
- Os tipos de estruturas de dados.
- Como declarar e manipular structs.
- Aninhamento de structs.
- Agregar structs em um vetor.

Explore +

Para saber mais sobre os assuntos tratados neste material:

- Pesquise na internet o canal Linguagem C - Programação Descomplicada.
- [Clique aqui](#) para baixar o código-fonte dos exemplos vistos ao longo do conteúdo.

Referências

SCHILD, H. **C, completo e total**. 3. ed. São Paulo: Makron Books, 1996.